
Desktop Scene Reconstruction

Plane-sweep depth, open-vocabulary segmentation, and image-based view synthesis for
metric-semantic understanding of a sparse multi-view capture

CP260-2026 — Robot Perception · Final Project Report

Vikas Rajpoot (27442)

Karlosh Yadav (27506)

Contents

1	Problem statement	2
2	Related work	2
2.1	Multi-view depth	2
2.2	Neural and explicit radiance fields	2
2.3	Open-vocabulary detection and segmentation	3
2.4	Image-based rendering	3
3	System architecture	3
3.1	Why this factorisation?	3
4	Implementation details	4
4.1	Plane-sweep depth	4
4.2	Semantic segmentation	4
4.3	Plane-RANSAC OBBs	4
4.4	Novel-view rendering	4
4.5	Output format	4
5	Experimental results	5
5.1	Depth estimation quality	5
5.2	Connector localisation	5
5.3	Novel-view synthesis	5
5.4	Runtime	5
6	Conclusions	5
7	References	6

1. Problem statement

We are given 16 RGB photographs of a desktop computer at 2560×1440 resolution together with the corresponding 4×4 camera-to-world transformations (a *poses.json* file). The task is two-fold:

- **View synthesis.** Render plausible RGB images at novel poses provided at evaluation time. Quality is judged by photometric reconstruction error.
- **Metric-semantic localisation.** Estimate oriented 3-D bounding boxes (centre, extent, rotation) for connectors visible on the back panel of the case — at minimum the power inlet and the RJ-45 ethernet socket, and additional connectors at the evaluator’s discretion (the bonus task).

The dataset is deliberately sparse — sixteen views of a single back panel, spanning a roughly 60° arc — which constrains the design space. Methods that require dense supervision or many hours of training (full NeRF, 3D Gaussian Splatting from scratch) are unlikely to converge cleanly on so few photographs. We therefore adopt a pipeline that exploits the supplied poses and treats geometry, semantics, and rendering as separate, well-conditioned subproblems.

2. Related work

2.1 Multi-view depth

Plane-sweep stereo (Collins, 1996; Gallup et al., 2007) is one of the oldest dense depth-estimation strategies for posed cameras. A reference image is compared against neighbours warped under fronto-parallel depth hypotheses; the depth that minimises patch dissimilarity is selected per pixel. Modern learned MVS networks (MVSNet, CasMVSNet, IterMVS) replace patch-NCC with a learned cost volume but still rely on the same warping primitive; with only 16 views and no fine-tuning data, classical plane-sweep with NCC is competitive and avoids any training step.

2.2 Neural and explicit radiance fields

NeRF (Mildenhall et al., 2020) and 3D Gaussian Splatting (Kerbl et al., 2023) are the current state of the art for photoreal novel-view synthesis from posed cameras. NeRF fits a fully-connected MLP that maps (position, direction) to colour and density and trains for several thousand iterations. 3DGS represents the scene as a collection of anisotropic Gaussians and rasterises them in real time after a few minutes of training. Both methods need either many images or a robust point-cloud initialisation to avoid floaters in low-overlap regions; with only 16 photographs the optimisation is ill-conditioned. We therefore use them as *references* rather than as our primary reconstruction.

2.3 Open-vocabulary detection and segmentation

Recent vision-language models (CLIP, OWL-ViT, Grounding DINO) detect arbitrary categories described by natural-language prompts. Segment-Anything (Kirillov et al., 2023) and its small variant MobileSAM (Zhang et al., 2023) take a 2-D box or point prompt and produce a pixel-precise mask. Used together, they provide zero-shot segmentation of named objects without any task-specific training data. We rely on this combination for the semantic stage of our pipeline.

2.4 Image-based rendering

Before learning-based methods became dominant, view synthesis was performed by warping pixels from neighbouring views into the target frame using estimated depth (Shum & Kang, 2000; Chaurasia et al., 2013). The output is then blended using view-distance-based weights. We use a forward-warping variant of this idea, since it is fast, deterministic, and integrates naturally with our depth maps.

3. System architecture

The pipeline factorises the problem into four loosely coupled stages, summarised in the table below. Each stage owns a single Python module under *src/*; the *driver.py* module sequences them.

Stage	Module	Inputs → outputs
Asset I/O	asset_io.py	Disk images, poses.json, intrinsic.json → numpy arrays
Depth estimation	depth_sweep.py	Images + K + poses → per-frame depth & confidence
Segmentation	vlm_segmenter.py	Images + text prompts → per-frame binary masks
OBB fitting	box_fitter.py	Masks + depth → 3-D bounding boxes
Rendering	view_render.py	Target pose + depth maps → synthesised RGB

3.1 Why this factorisation?

Each module solves a sub-problem for which a well-understood, almost-classical, non-learned solution exists. This matters for two reasons:

- It makes failure modes easy to localise — depth issues do not silently corrupt segmentation and *vice-versa*.
- The compute budget on a free-tier T4 cannot accommodate any stage that requires training a neural network from scratch.

Open-vocabulary detection and Segment-Anything are the only learned components, and they are used purely at inference time with publicly released weights.

4. Implementation details

4.1 Plane-sweep depth

For a reference frame we choose four neighbours by camera-centre proximity and perform an inverse-depth sweep over $L = 96$ levels between 0.05 m and 1.20 m. Each level induces a homography $H = K(R - tn^T/d)K^{-1}$. The reference is compared to each warped neighbour with a $(2r + 1) \times (2r + 1)$ NCC-style cost computed via integral images, giving an $L \times H \times W$ cost volume. The argmin per pixel yields the depth, and the ratio between the best and second-best costs is used as a confidence map. A 5×5 median filter cleans the depth edges without further blurring.

4.2 Semantic segmentation

We feed each prompt — for example ‘an RJ45 ethernet port on a computer back panel’ — to OWL-ViT v2 (base, patch-16) and keep the highest-scoring box per frame. Each box is then refined into a tight binary mask with MobileSAM (*vit_t* backbone, ~ 10 M parameters, ~ 40 MB of weights), which is much lighter than SAM-H and runs comfortably on a T4. Frames where no detection clears the score threshold are simply skipped — there is no need for hand-labelled boxes.

4.3 Plane-RANSAC OBBs

For every entity we collect the masked pixels across all detected frames, back-project them through the corresponding depth map, and assemble a single world-frame point cloud. A plane RANSAC (1500 iterations, 4 mm inlier threshold) estimates the back-panel surface normal. Using that normal as one OBB axis, we perform PCA on the in-plane projections to recover the major and minor in-plane directions. Box extents come from the 2nd / 98th percentiles of the projected coordinates (so the box is robust to a few outlier pixels), and the third extent is the physical protrusion of a typical motherboard connector — a 6 mm prior. The resulting rotation matrix is canonicalised so that the panel-normal axis points outward in world space.

4.4 Novel-view rendering

For a novel target pose we rank all source frames by a combined translation-and-direction score, keep the five closest, and forward-warp each of them into the target view using its depth map. Pixels are blended with a Gaussian weight in the camera-centre distance ($\sigma = 4$ cm); a small dilation fills 1-pixel disocclusion holes. This image-based formulation cannot fully match the photometric quality of a NeRF that has been trained for many hours, but it produces a convincing rendering in seconds and remains entirely deterministic.

4.5 Output format

Each entity is written to *outputs/answers.json* with the convention $\text{corner}_i = \text{center} + R^T \cdot (\text{sign}_i \cdot \text{extent})$, matching the format the course evaluator expects.

5. Experimental results

5.1 Depth estimation quality

Depth maps are smooth on the planar back panel and degrade gracefully on the speaker / cable areas where the camera baseline is not well-suited to MVS. Median per-pixel confidence on the panel itself is ~ 0.45 , dropping to ~ 0.10 in specular highlights of the metal heatsink. Because the OBB stage uses confidence thresholding (> 0.05), the unreliable regions are filtered out automatically before plane fitting.

5.2 Connector localisation

The pipeline correctly localises the four entities asked at evaluation time:

Entity	centre (m)	extent (m)	Detected in N frames
power_socket	(0.271, 0.206, 0.542)	(0.029, 0.018, 0.006)	5–6
ethernet_socket	(0.166, 0.125, 0.856)	(0.020, 0.013, 0.006)	4–6
hdmi_socket_left	(0.248, 0.233, 0.838)	(0.009, 0.006, 0.006)	3–4
usb_socket_top_right	(0.180, 0.129, 0.898)	(0.016, 0.008, 0.006)	3–4

Centre coordinates are in the world frame defined by *poses.json*; extents are *half*-edge lengths (matching the GT convention). The centre of the VGA socket — used as a sanity check during development — agreed with the value released alongside the dataset to within roughly 6 mm.

5.3 Novel-view synthesis

Forward-warping with five source views produces sharp panel detail close to the original camera ring (PSNR ≈ 22 dB measured on held-out frame 471) and graceful softening at large baseline excursions. Disocclusions appear as small black bands that the dilation pass partially fills.

5.4 Runtime

- Plane-sweep depth (16 frames @ 2560×1440, 96 levels): ~ 12 minutes on a T4.
- OWL-ViT detection across 16 frames $\times$ 5 prompts: ~ 25 seconds.
- MobileSAM segmentation: ~ 15 seconds.
- OBB fitting + RANSAC: < 1 second.
- Total wall-clock for a complete fresh run: ≈ 14 minutes.

6. Conclusions

Decoupling the problem into a classical plane-sweep depth stage, an open-vocabulary segmentation stage, and a robust plane-fit OBB stage gives an end-to-end pipeline that is fast, determin-

istic, and easy to reason about. The only learned components are off-the-shelf vision–language models used at inference time, which makes the pipeline portable across new scenes without any fine-tuning. The OBB convention matches the course evaluator’s format directly, so adding new entities at evaluation time only requires extending a prompt dictionary by one line.

We see the following directions for follow-up work:

- Replace the constant 6 mm protrusion prior with a photometric refinement that searches along the panel normal for an actual change in colour gradient.
- Use slanted-plane sweeps to handle the side panels, where the fronto-parallel assumption is least accurate.
- Initialise a 3-D Gaussian Splat from the dense MVS point cloud and run a brief (~ 2000 -iteration) refinement to bridge the remaining gap to NeRF-quality view synthesis without paying the full training cost.

7. References

- Collins, R. T. *A space-sweep approach to true multi-image matching*. CVPR 1996.
- Gallup, D. et al. *Real-time plane-sweeping stereo with multiple sweeping directions*. CVPR 2007.
- Mildenhall, B. et al. *NeRF: Representing scenes as neural radiance fields for view synthesis*. ECCV 2020.
- Kerbl, B. et al. *3D Gaussian Splatting for real-time radiance field rendering*. SIGGRAPH 2023.
- Minderer, M. et al. *Simple open-vocabulary object detection (OWL-ViT)*. ECCV 2022.
- Kirillov, A. et al. *Segment Anything*. ICCV 2023.
- Zhang, C. et al. *Faster Segment Anything: Towards lightweight SAM for mobile applications*. arXiv 2306.14289, 2023.
- Shum, H.-Y. and Kang, S. B. *A review of image-based rendering techniques*. SPIE Visual Communications and Image Processing 2000.
- Chaurasia, G. et al. *Depth synthesis and local warps for plausible image-based navigation*. ACM TOG 2013.